

EAD 01 — Preparação do Ambiente de Desenvolvimento

FALECT-CC-035 — Processamento de Alto Desempenho

Esta atividade prepara o computador que será utilizado nas práticas da disciplina. Ao final, o estudante deverá possuir um ambiente Linux funcional, Visual Studio Code configurado, compiladores e ferramentas de desenvolvimento instalados, Git/GitHub configurados, OpenMP e MPI testados e, quando disponível para sua conta, assistentes de programação por IA configurados no VS Code.

Atividade individual.

Entregável: um único arquivo chamado `diagnostico-ambiente-pad.txt`.

O arquivo deverá conter informações do sistema e do hardware, versões das ferramentas instaladas e as saídas dos testes OpenMP e MPI.

Privacidade: envie o diagnóstico somente pelo sistema acadêmico. Não faça commit desse arquivo nem o publique em GitHub, chat ou fórum.

Uso de IA: IA permitida apenas como apoio para compreender instruções e diagnosticar erros. Não envie o arquivo de diagnóstico a uma ferramenta de IA e não aceite comandos sem revisá-los.

Prazo e critérios: consulte a atividade no SIGAA. Será verificada a presença das seções solicitadas, das versões das ferramentas e das saídas dos testes OpenMP e MPI; limitações justificadas devem ser registradas.

1. Leia antes de começar

Esta atividade foi escrita para estudantes que ainda não têm familiaridade com Linux, terminal, compiladores ou VS Code. Portanto:

1. siga os passos na ordem apresentada;
2. leia a explicação antes de executar o comando;
3. execute um bloco de cada vez;
4. se aparecer erro, leia a mensagem antes de tentar outra coisa;
5. não instale programas aleatórios encontrados na Internet para “resolver” o problema;
6. anote erros que não conseguir resolver;
7. não habilite serviços pagos nem forneça cartão de crédito;
8. não execute comandos sugeridos por IA sem antes lê-los e tentar compreender o que fazem.

1.1. Aviso importante para quem já utiliza Linux

Se o computador já utiliza **Linux nativamente**, não instale WSL.

Ubuntu ou outra distribuição baseada em Debian

Você deve:

- **pular as Seções 2, 3 e 4**, que tratam de Windows e WSL;
- iniciar na **Seção 5 — Atualizar o Linux**;
- instalar a versão Linux do VS Code na Seção 6;
- **pular a Seção 7 — extensão WSL**;
- continuar normalmente a partir da criação do diretório de trabalho.

Fedora, Arch Linux, openSUSE ou outra distribuição

Você também deve pular o WSL. Entretanto, os comandos de instalação de pacotes serão diferentes de apt.

A atividade foi padronizada para Ubuntu/Debian. Nesse caso:

- utilize o gerenciador de pacotes oficial da sua distribuição;
- instale ferramentas equivalentes às listadas nesta atividade;
- não altere os códigos de teste;
- o entregável continua sendo o mesmo;
- informe no campo textual da entrega qual distribuição foi utilizada.

macOS

A atividade foi preparada para Windows + WSL ou Linux. Consulte o professor antes de adaptar o ambiente.

2. Usuários de Windows — verificar a versão

Usuários de Linux nativo: pule para a Seção 5.

Pressione Win + R, digite:

winver

O método atual de instalação simplificada do WSL é suportado no Windows 10 versão 2004, Build 19041 ou posterior, e no Windows 11.

Documentação oficial:

<https://learn.microsoft.com/pt-br/windows/wsl/install>

3. Usuários de Windows — instalar ou verificar WSL 2

Usuários de Linux nativo: pule esta seção.

3.1. Verificar se já existe

Abra o PowerShell e execute:

```
wsl --status
```

Depois:

```
wsl --list --verbose
```

Uma saída possível é:

NAME	STATE	VERSION
* Ubuntu	Stopped	2

O nome e o estado podem variar. O importante é a coluna VERSION indicar 2.

Se já existir Ubuntu com WSL 2, **não reinstale**. Prossiga para a Seção 4.

3.2. Instalar quando necessário

Abra **PowerShell como Administrador** e execute:

```
wsl --install
```

Reinicie o computador se solicitado.

Depois confira:

```
wsl --list --verbose
```

Novas instalações realizadas por esse procedimento utilizam WSL 2 por padrão.

3.3. Caso o Ubuntu não seja instalado automaticamente

Liste distribuições:

```
wsl --list --online
```

Se Ubuntu-24.04 estiver disponível, por exemplo:

```
wsl --install -d Ubuntu-24.04
```

Use exatamente um nome apresentado por `wsl --list --online`.

4. Usuários de Windows — primeiro acesso ao Ubuntu

Usuários de Linux nativo: pule esta seção.

Abra **Ubuntu** no menu Iniciar.

Na primeira execução será solicitado:

- um nome de usuário Linux;
- uma senha para esse usuário.

Essa conta é diferente da conta do Windows.

Atenção à senha

Quando o Linux solicita uma senha após `sudo`, nenhum caractere aparece enquanto você digita. Isso é normal.

Digite a senha e pressione Enter.

Verifique o sistema:

```
uname -srm
```

Em WSL 2 a saída normalmente contém referências a Linux, Microsoft e WSL2. Os textos exatos variam.

Depois:

```
cat /etc/os-release
```

Você deve encontrar uma identificação da distribuição semelhante a:

```
NAME="Ubuntu"
PRETTY_NAME="Ubuntu ... LTS"
```

5. Atualizar Ubuntu/Linux

A partir deste ponto, usuários de Windows + WSL e usuários de Ubuntu Linux nativo seguem os mesmos passos.

Execute:

```
sudo apt update
```

Confira as atualizações disponíveis:

```
apt list --upgradable
```

Em computador pessoal, você pode aplicar as atualizações com `sudo apt upgrade` depois de revisar a lista. Em laboratório ou equipamento administrado, não faça uma atualização geral sem autorização do responsável.

Instale utilitários básicos:

```
sudo apt install -y curl wget ca-certificates
```

Os comandos apt exigem conexão com a Internet.

6. Instalar Visual Studio Code

Windows + WSL

Instale o VS Code **no Windows**, e não dentro do Ubuntu.

<https://code.visualstudio.com/>

Durante a instalação, quando disponível, marque **Add to PATH**. Isso permitirá abrir uma pasta Linux usando `code ..`

Documentação:

<https://learn.microsoft.com/pt-br/windows/wsl/tutorials/wsl-vscode>

Linux nativo

Baixe a versão apropriada no site oficial:

<https://code.visualstudio.com/>

No Ubuntu/Debian, prefira o pacote oficial `.deb`.

Depois confirme:

```
code --version
```

A saída mostrará uma versão, por exemplo:

```
1.xx.x
```

A versão exata mudará ao longo do tempo.

7. Windows + WSL — instalar a extensão WSL

Linux nativo: pule esta seção.

No VS Code pressione:

Ctrl + Shift + X

Pesquise:

WSL

Instale a extensão oficial **WSL**, publicada pela Microsoft.

Ela permite que a interface do VS Code funcione no Windows enquanto compiladores, terminal e arquivos trabalham dentro do Linux/WSL.

8. Criar o único diretório da atividade

Todos os arquivos desta atividade deverão ficar na mesma pasta.

Execute:

```
mkdir -p ~/unemat/pad/ead01-ambiente  
cd ~/unemat/pad/ead01-ambiente
```

Confira:

```
pwd
```

A saída deverá ser semelhante a:

```
/home/seu_usuario/unemat/pad/ead01-ambiente
```

Ao final, a pasta deverá conter:

```
ead01-ambiente/  
├── teste_openmp.c  
├── teste_openmp  
├── teste_mpi.c  
├── teste_mpi  
└── diagnostico-ambiente-pad.txt
```

9. Abrir a pasta no VS Code

Ainda no diretório da atividade:

```
code .
```

Windows + WSL

Na primeira execução, o VS Code poderá instalar o VS Code Server no WSL. Aguarde. Verifique se a janela indica conexão ao WSL/Ubuntu.

Linux nativo

A pasta deverá abrir diretamente.

Abrir o terminal integrado

No VS Code escolha **Terminal > New Terminal** ou utilize:

Ctrl + `

Confirme:

```
pwd
```

O caminho deve terminar em:

```
/unemat/pad/ead01-ambiente
```

A partir daqui, faça o restante da atividade no VS Code.

10. Instalar compiladores e ferramentas

No terminal integrado do VS Code:

```
sudo apt install -y \  
  build-essential \  
  gcc \  
  g++ \  
  gfortran \  
  make \  
  cmake \  
  gdb \  
  git \  
  pkg-config \  
  openssh-client \  
  htop \  
  hwloc \  
  numactl \  
  pciutils
```

Ferramentas principais:

- gcc: compilador C;
 - g++: compilador C++;
 - gfortran: compilador Fortran;
 - make: automação de compilação;
 - cmake: configuração de builds;
 - gdb: depuração;
 - git: controle de versão;
 - openssh-client: acesso SSH, usado posteriormente nos servidores e no cluster;
 - htop: inspeção de processos/CPU;
 - hwloc: inspeção da topologia do hardware;
 - numactl: informações relacionadas a NUMA;
 - pciutils: inclui lspci, útil para identificar dispositivos como GPUs.
-

11. Verificar as ferramentas

Execute separadamente:

```
gcc --version
```

Primeira linha semelhante a:

```
gcc (Ubuntu ...) 13.x.x
```

Depois:

```
g++ --version
gfortran --version
make --version
cmake --version
gdb --version
```

As versões exatas podem ser diferentes. O importante é que os comandos existam e executem sem command not found.

12. Instalar Python

```
sudo apt install -y python3 python3-pip python3-venv
```

Verifique:

```
python3 --version
pip3 --version
```

Saída típica:

```
Python 3.x.x
```

13. Instalar Java

```
sudo apt install -y default-jdk
```

Verifique:

```
java -version
javac -version
```

O segundo comando deverá mostrar algo semelhante a:

```
javac ...
```

14. Instalar Julia

Execute:

```
curl -fsSL https://install.julialang.org | sh
```

Siga as instruções do instalador. Depois feche e abra novamente o terminal integrado; se necessário:

```
source ~/.bashrc
```

Verifique:

```
julia --version
```

Saída semelhante a:

```
julia version 1.x.x
```

Documentação oficial:

<https://julialang.org/install/>

15. Instalar OpenMPI

```
sudo apt install -y openmpi-bin libopenmpi-dev
```

Verifique:

```
mpicc --version  
mpirun --version
```

A saída do segundo deve conter algo semelhante a:

```
mpirun (Open MPI) ...
```

16. Extensões recomendadas do VS Code

Abra Extensions com Ctrl + Shift + X e instale:

- **C/C++** — Microsoft;
- **C/C++ Extension Pack** — Microsoft;
- **Python** — Microsoft;
- **Pylance** — Microsoft;
- **Extension Pack for Java** — Microsoft;
- **Julia** — Julia Language Support;
- **Modern Fortran**;
- **GitLens**;
- **Error Lens**;
- **Markdown All in One**.

Em Windows + WSL, algumas extensões podem oferecer **Install in WSL**. Quando o VS Code indicar que a extensão precisa estar no ambiente WSL, utilize essa opção.

17. Primeiro programa OpenMP — criar no VS Code

Nesta atividade, os programas serão criados diretamente pelo VS Code, e não com nano.

17.1. Criar teste_openmp.c

No Explorer do VS Code:

1. clique em **New File / Novo Arquivo**;
2. crie teste_openmp.c;
3. cole:

```
#include <stdio.h>  
#include <omp.h>  
  
int main(void) {  
    #pragma omp parallel  
    {  
        printf("Ola da thread %d de %d\n",  
               omp_get_thread_num(),  
               omp_get_num_threads());  
    }  
}
```



```
    return 0;
}
```

Salve com Ctrl + S.

17.2. Compilar

No terminal integrado:

```
gcc -O2 -Wall -Wextra -fopenmp teste_openmp.c -o teste_openmp
```

Se estiver correto, normalmente o compilador **não imprime nenhuma mensagem** e apenas retorna ao prompt.

Confirme o executável:

```
ls -l teste_openmp
```

17.3. Executar com quatro threads

```
OMP_NUM_THREADS=4 ./teste_openmp
```

Uma saída possível:

```
Ola da thread 0 de 4
Ola da thread 2 de 4
Ola da thread 1 de 4
Ola da thread 3 de 4
```

A ordem pode mudar entre execuções. Isso é esperado.

Devem aparecer exatamente as threads 0, 1, 2 e 3, todas com de 4.

18. Primeiro programa MPI — criar no VS Code

18.1. Criar teste_mpi.c

No Explorer crie:

teste_mpi.c

Cole:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    int rank;
    int size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Ola do processo %d de %d\n", rank, size);

    MPI_Finalize();
    return 0;
}
```

Salve.

18.2. Compilar

```
mpicc -O2 -Wall -Wextra teste_mpi.c -o teste_mpi
```

A compilação correta normalmente termina sem mensagem.

Confirme:

```
ls -l teste_mpi
```

18.3. Executar com quatro processos

```
mpirun --oversubscribe -np 4 ./teste_mpi
```

Saída possível:

```
Ola do processo 0 de 4
Ola do processo 1 de 4
Ola do processo 2 de 4
Ola do processo 3 de 4
```

A ordem pode variar. Devem existir os processos 0, 1, 2 e 3.

19. Criar ou verificar conta pessoal no GitHub

Caso ainda não tenha:

<https://github.com/signup>

Recomendação importante

Mantenha na conta um e-mail ao qual você continuará tendo acesso depois de concluir o curso.

Não recomendamos criar uma conta GitHub cuja continuidade dependa exclusivamente do e-mail institucional da UNEMAT.

Se já possui conta pessoal, utilize a existente. Não crie outra apenas para a disciplina.

Escolha um nome de usuário adequado para futura identificação acadêmica/profissional.

Como os commits e os forks podem ser públicos, ative a opção **Keep my email addresses private** nas configurações de e-mail do GitHub. A plataforma mostrará um endereço noreply próprio para uso nos commits.

Documentação:

<https://docs.github.com/pt/get-started/start-your-journey/creating-an-account-on-github>

20. Configurar o Git

Nesta primeira atividade **não haverá clone, fork, push ou Pull Request**. O uso de repositórios será trabalhado na próxima atividade.

O nome configurado aparecerá publicamente nos commits. Use o nome público escolhido para o GitHub; a associação com sua identidade acadêmica será informada privadamente pelo SIGAA. Não inclua matrícula ou documento:

```
git config --global user.name "SEU NOME PUBLICO"
```

Configure o endereço noreply exibido pelo GitHub em **Settings > Emails**:

```
git config --global user.email "ENDERECO-NOREPLY-EXIBIDO-PELO-GITHUB"
```

Confira:

```
git config --global user.name
git config --global user.email
git --version
```

21. Configurar GitHub Copilot

O GitHub Copilot possui uma modalidade **Copilot Free**, com uso limitado, além de modalidades para estudantes elegíveis e planos pagos.

Não é necessário contratar um plano pago para realizar esta atividade.

Planos:

<https://docs.github.com/pt/copilot/get-started/plans>

Configuração no VS Code

Use sua **conta pessoal do GitHub**.

No VS Code:

1. localize o ícone do Copilot/recursos de IA;
2. escolha **Use AI Features / Usar recursos de IA**;
3. faça login com GitHub;
4. conclua as autorizações apresentadas.

Nas versões atuais do VS Code, as extensões necessárias do Copilot são configuradas automaticamente. Se a conta ainda não tiver plano, o VS Code pode oferecer Copilot Free.

Documentação:

<https://code.visualstudio.com/docs/setup/copilot>

Teste simples

Abra teste_openmp.c e, no chat do Copilot, digite:

Explique, sem modificar o código, o que o arquivo teste_openmp.c faz.

Se responder, considere o Copilot funcional.

Se não estiver disponível, registre posteriormente:

GitHub Copilot: não configurado

ou:

GitHub Copilot: sem acesso na conta

Isso não invalida a atividade.

22. Configurar Gemini Code Assist

Aviso importante

O modelo de acesso do Gemini Code Assist mudou em 2026. O acesso das extensões do IDE para contas pessoais foi descontinuado em junho de 2026.

Nesta disciplina, tente a configuração com a **conta institucional da UNEMAT**, quando essa conta possuir o acesso organizacional necessário ao Gemini Code Assist Standard ou Enterprise.

Ter uma conta Google institucional, por si só, não garante que o serviço tenha sido habilitado para a organização.

Não habilite faturamento. Não contrate plano pago. Não cadastre cartão de crédito.

Se o acesso institucional não estiver habilitado, registre isso e prossiga normalmente.

Documentação:

<https://developers.google.com/gemini-code-assist/docs/deprecations/code-assist-individuals?hl=pt-BR>

<https://docs.cloud.google.com/gemini/docs/codeassist/set-up-gemini?hl=pt-BR>

Instalar no VS Code

1. pressione Ctrl + Shift + X;
2. pesquise Gemini Code Assist;
3. instale a extensão oficial do Google;
4. recarregue o VS Code se solicitado;
5. abra o painel Gemini Code Assist;
6. inicie a autenticação disponibilizada pela extensão;
7. utilize sua conta institucional da UNEMAT;
8. se for solicitado projeto Google Cloud, utilize apenas um projeto fornecido institucionalmente.

Teste simples

Abra `teste_mpi.c` e solicite:

Explique, sem modificar o código, a diferença entre rank e size neste programa MPI.

Se responder:

Gemini Code Assist: funcionando com conta institucional

Se não houver acesso:

Gemini Code Assist: sem acesso institucional

Isso não invalida a atividade.

23. Informações de hardware importantes para a disciplina

O diagnóstico final coletará informações úteis para Processamento de Alto Desempenho:

- arquitetura do processador;
- modelo da CPU;
- quantidade de CPUs lógicas;
- núcleos por socket;
- threads por núcleo;

- caches, quando reportados;
- memória RAM;
- topologia NUMA;
- GPU, quando detectável;
- versões de compiladores;
- ambiente WSL ou Linux nativo.

Antes de gerar o arquivo final:

```
pwd
ls -la
```

Você deve estar em:

~/unemat/pad/ead01-ambiente

E devem existir:

```
teste_openmp.c
teste_openmp
teste_mpi.c
teste_mpi
```

24. Gerar o arquivo diagnostico-ambiente-pad.txt

Copie **todo o bloco abaixo** no terminal integrado, dentro do diretório da atividade:

```
{
echo "=====
echo "FALECT-CC-035 - PROCESSAMENTO DE ALTO DESEMPENHO"
echo "EAD 01 - DIAGNOSTICO DO AMBIENTE"
echo "=====
echo
echo "Data da coleta: $(date '+%Y-%m-%d') "
echo

echo "===== TIPO DE AMBIENTE ====="
if grep -qi microsoft /proc/version 2>/dev/null; then
    echo "Ambiente: Windows + WSL"
else
    echo "Ambiente: Linux nativo"
fi
echo

echo "===== SISTEMA OPERACIONAL ====="
uname -srm
grep '^PRETTY_NAME=' /etc/os-release 2>/dev/null || true
echo

echo "===== PROCESSADOR ====="
cpu_fields='Architecture|CPU\(\s\)|On-line CPU\(\s\) list|'
cpu_fields+='Vendor ID|Model name|Socket\(\s\)|'
cpu_fields+='Core\(\s\) per socket|Thread\(\s\) per core|'
cpu_fields+='CPU max MHz|L1d cache|L1i cache|L2 cache|'
cpu_fields+='L3 cache|NUMA node\(\s\)|'
LC_ALL=C lscpu | grep -E "$cpu_fields" || true
echo
```

```

echo "===== MEMORIA ====="
free -h
echo

echo "===== NUMA ====="
numactl --hardware 2>/dev/null || echo "Informacao NUMA nao disponivel."
echo

echo "===== GPU / VIDEO ====="
if command -v nvidia-smi >/dev/null 2>&1; then
    echo "--- NVIDIA detectada ---"
    nvidia-smi --query-gpu=name,memory.total,driver_version \
        --format=csv,noheader 2>/dev/null || nvidia-smi
else
    echo "nvidia-smi nao encontrado."
fi

echo
echo "--- Dispositivos graficos detectados pelo Linux ---"
lspci 2>/dev/null | grep -Ei 'vga|3d|display' \
    || echo "Nenhum adaptador grafico identificado pelo lspci."

if grep -qi microsoft /proc/version 2>/dev/null \
    && command -v powershell.exe >/dev/null 2>&1; then
    echo
    echo "--- GPU identificada pelo Windows host ---"
    gpu_query='Get-CimInstance Win32_VideoController'
    gpu_format='Select-Object Name,DriverVersion | Format-Table -AutoSize'
    powershell.exe -NoProfile -Command "$gpu_query | $gpu_format" \
        2>/dev/null | tr -d '\r' || true

    echo
    echo "--- Memoria RAM fisica do Windows host (GB) ---"
    ram_query='$m=(Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory'
    ram_calc='[math]::Round($m/1GB,2)'
    powershell.exe -NoProfile -Command "$ram_query; $ram_calc" \
        2>/dev/null | tr -d '\r' || true
fi
echo

echo "===== COMPILADORES ====="
gcc --version 2>/dev/null | head -n 1
g++ --version 2>/dev/null | head -n 1
gfortran --version 2>/dev/null | head -n 1
echo

echo "===== BUILD / DEBUG ====="
make --version 2>/dev/null | head -n 1
cmake --version 2>/dev/null | head -n 1
gdb --version 2>/dev/null | head -n 1
echo

echo "===== PYTHON ====="
python3 --version 2>&1
pip3 --version 2>&1

```

```

echo

echo "===== JAVA ====="
java -version 2>&1 | head -n 1
javac -version 2>&1
echo

echo "===== JULIA ====="
julia --version 2>&1
echo

echo "===== MPI ====="
mpirun --version 2>&1 | head -n 2
echo

echo "===== GIT ====="
git --version 2>&1
echo

echo "===== VS CODE ====="
code --version 2>&1 | head -n 1
echo

echo "===== TESTE OPENMP ====="
if [ -x ./teste_openmp ]; then
    OMP_NUM_THREADS=4 ./teste_openmp
else
    echo "ERRO: executavel teste_openmp nao encontrado."
fi
echo

echo "===== TESTE MPI ====="
if [ -x ./teste_mpi ]; then
    mpirun --oversubscribe -np 4 ./teste_mpi
else
    echo "ERRO: executavel teste_mpi nao encontrado."
fi
echo

echo "===== ARQUIVOS DA ATIVIDADE ====="
find . -maxdepth 1 -type f -printf '%f\n' | sort
echo

echo "===== FIM DO DIAGNOSTICO ====="
} 2>&1 | tee diagnostico-ambiente-pad.txt

```

25. Conferir a saída esperada

Abra diagnostico-ambiente-pad.txt pelo Explorer do VS Code.

Ele deve conter seções como:

```

TIPO DE AMBIENTE
SISTEMA OPERACIONAL
PROCESSADOR

```

MEMORIA
NUMA
GPU / VIDEO
COMPILADORES
BUILD / DEBUG
PYTHON
JAVA
JULIA
MPI
GIT
VS CODE
TESTE OPENMP
TESTE MPI
ARQUIVOS DA ATIVIDADE

CPU

Exemplo de formato:

Architecture: x86_64
CPU(s): 8
Model name: ...
Thread(s) per core: 2
Core(s) per socket: 4

Os valores serão diferentes em cada computador.

Memória

Formato semelhante a:

	total	used	free	shared	buff/cache	available
Mem:	7.7Gi	...				
Swap:	...					

No WSL, a memória visível ao Linux pode ser diferente da RAM física total do computador. Por isso o script também tenta consultar a memória do Windows host.

GPU

Se houver NVIDIA e nvidia-smi estiver disponível, poderá aparecer modelo, memória e driver.

Se não houver identificação, isso **não invalida a atividade**. O computador pode possuir GPU integrada ou o ambiente pode não expor a informação da mesma forma.

OpenMP

Devem existir quatro linhas com as threads 0, 1, 2 e 3, todas de 4.

MPI

Devem existir quatro linhas com os processos 0, 1, 2 e 3, todos de 4.

26. Registrar o estado dos assistentes de IA

Abra diagnostico-ambiente-pad.txt no VS Code e **não apague a saída automática**.

Ao final, acrescente:

```
===== ASSISTENTES DE IA =====  
Gemini Code Assist: INFORME O STATUS  
GitHub Copilot: INFORME O STATUS
```

Exemplos válidos:

```
Gemini Code Assist: funcionando com conta institucional  
GitHub Copilot: Copilot Free funcionando
```

ou:

```
Gemini Code Assist: sem acesso institucional  
GitHub Copilot: não configurado
```

Salve com Ctrl + S.

27. Conferência final da pasta

Execute:

```
pwd  
ls -lh
```

A pasta deve ser:

```
~/unemat/pad/ead01-ambiente
```

E deve conter pelo menos:

```
teste_openmp.c  
teste_openmp  
teste_mpi.c  
teste_mpi  
diagnostico-ambiente-pad.txt
```

28. Checklist

Sistema

- ☐ Windows + WSL 2 ou Linux nativo funcionando.
- ☐ Ubuntu/Linux inicia normalmente.
- ☐ sudo apt update funciona.

VS Code

- ☐ VS Code instalado.
- ☐ Pasta ~/unemat/pad/ead01-ambiente abre no VS Code.
- ☐ Terminal integrado funciona.
- ☐ Em Windows, VS Code está conectado ao WSL.

Ferramentas

- ☐ gcc --version funciona.
- ☐ g++ --version funciona.
- ☐ gfortran --version funciona.
- ☐ make --version funciona.
- ☐ cmake --version funciona.
- ☐ gdb --version funciona.
- ☐ python3 --version funciona.
- ☐ java -version funciona.
- ☐ javac --version funciona.
- ☐ julia --version funciona.
- ☐ mpirun --version funciona.

Programas

- ☐ teste_openmp.c criado no VS Code.
- ☐ teste_openmp compilado.
- ☐ OpenMP executado com quatro threads.
- ☐ teste_mpi.c criado no VS Code.
- ☐ teste_mpi compilado.
- ☐ MPI executado com quatro processos.

Git/GitHub

- ☐ Possui conta pessoal no GitHub.
- ☐ A conta usa um e-mail que continuará acessível após o curso.
- ☐ A privacidade de e-mail está ativada no GitHub.
- ☐ Nome do Git configurado.
- ☐ E-mail noreply do GitHub configurado no Git.

IA

- ☐ GitHub Copilot verificado.
- ☐ Gemini Code Assist instalado/verificado.
- ☐ Estado de ambos registrado no TXT.

Entregável

- ☐ diagnostico-ambiente-pad.txt existe.
 - ☐ Contém informações de hardware.
 - ☐ Contém versões das ferramentas.
 - ☐ Contém saída do OpenMP.
 - ☐ Contém saída do MPI.
 - ☐ Contém estado dos assistentes de IA.
-

29. O que entregar no sistema acadêmico

Envie **somente**:

diagnostico-ambiente-pad.txt

Não envie executáveis, códigos .c, imagens, ZIP, PDF ou Word.

Não adicione o diagnóstico ao fork da disciplina. O .gitignore do repositório o bloqueia por padrão.

Campo textual da entrega

Informe:

Nome completo:

Usuário do GitHub:

Ambiente utilizado: Windows + WSL 2 / Linux nativo

Distribuição Linux:

Observações:

Exemplo:

Nome completo: Maria de Souza Silva

Usuário do GitHub: mariasilva

Ambiente utilizado: Windows + WSL 2

Distribuição Linux: Ubuntu 24.04 LTS

Observações: Gemini Code Assist sem acesso institucional.

Nunca informe senha do GitHub, Google, UNEMAT, Windows ou Linux.

O professor não solicitará senha nesta atividade.

30. Problemas comuns

wsl --install mostra apenas ajuda

```
wsl --list --online
```

Depois instale uma distribuição usando exatamente o nome apresentado.

WSL aparece como VERSION 1

```
wsl --list --verbose
```

Depois, substituindo Ubuntu pelo nome correto:

```
wsl --set-version Ubuntu 2
```

code . não funciona no WSL

Confira:

- VS Code instalado no Windows;
- opção Add to PATH;
- extensão WSL instalada;
- terminal fechado e aberto novamente.

sudo não mostra a senha

É normal. Digite e pressione Enter.

command not found

A ferramenta provavelmente não foi instalada corretamente. Volte à seção de instalação correspondente.

Compilação não mostra mensagem

Isso normalmente significa sucesso. Confirme se o executável foi criado com `ls -l`.

OpenMP ou MPI mostra linhas em ordem diferente

Isso é esperado em execução paralela. Não tente “corrigir” a ordem.

nvidia-smi: command not found

Não invalida a atividade. Pode não existir GPU NVIDIA ou o ambiente pode não oferecer essa ferramenta.

Gemini não permite autenticação

Não habilite faturamento. Registre Gemini Code Assist: sem acesso institucional e continue.

Copilot pede plano

Não compre plano para esta atividade. Verifique Copilot Free ou, se elegível, modalidade estudantil.

31. Documentação oficial

WSL

<https://learn.microsoft.com/pt-br/windows/wsl/install>

<https://learn.microsoft.com/pt-br/windows/wsl/setup/environment>

VS Code + WSL

<https://learn.microsoft.com/pt-br/windows/wsl/tutorials/wsl-vscode>

<https://code.visualstudio.com/docs/remote/wsl>

VS Code

<https://code.visualstudio.com/>

GitHub

<https://docs.github.com/pt/get-started/start-your-journey/creating-an-account-on-github>

<https://docs.github.com/pt/get-started/git-basics>

GitHub Copilot

<https://code.visualstudio.com/docs/setup/copilot>

<https://docs.github.com/pt/copilot/get-started/plans>

Julia

<https://julialang.org/install/>

Gemini Code Assist

<https://developers.google.com/gemini-code-assist/docs/deprecations/code-assist-individuals?hl=pt-BR>

<https://docs.cloud.google.com/gemini/docs/codeassist/set-up-gemini?hl=pt-BR>

32. Vídeo complementar — Git e GitHub

Curso em Vídeo — Git e GitHub:

https://www.youtube.com/playlist?list=PLHz_AreHm4dm7ZULPAmadvNhH6vk9oNZA

O conteúdo de fork, clone, commit, push e Pull Request será trabalhado em uma atividade posterior.

33. Por que esta atividade é importante?

Em Processamento de Alto Desempenho, não basta fazer um programa “funcionar”. Ao longo da disciplina será necessário:

- compilar programas;
- utilizar diferentes linguagens;
- usar OpenMP e MPI;
- medir tempos;
- comparar versões sequenciais e paralelas;
- identificar núcleos e threads;
- observar memória e hardware;
- acessar servidores Linux;
- trabalhar posteriormente com um cluster;
- registrar experimentos de forma reproduzível.

O arquivo `diagnostico-ambiente-pad.txt` também permitirá compreender, mais adiante, por que dois estudantes podem obter tempos diferentes executando o mesmo programa em máquinas com arquiteturas e recursos diferentes.